جزوه‌ی هفته‌ی ۱: مرور پایتون و محیط کار

از لاگ خام پرواز تا پاسخ‌های مهندسی

هوش مصنوعی برای مهندسی هوافضا و مکانیک (دکتری)، بخش صفر

> **منشأ محتوا:** محتوای جدید. جزوه‌ی مدرس با موضوع «راه‌اندازی محیط و مقدمه‌ی یادگیری ماشین» هنگام نگارش در دسترس نبود و باید پس از دریافت با آن هماهنگ شود. مطالب ابزارهای پایتون با بازنویسی (نه رونویسی) از W3Schools (مقدمه‌ی NumPy، خواندن CSV و تحلیل DataFrame در Pandas، Pyplot در Matplotlib، مقدمه‌ی Git) و GeeksforGeeks (محیط مجازی پایتون، محیط مجازی با Anaconda، شروع کار با Jupyter، متد describe) تهیه شده است. روابط پرواز دورزدن برگرفته از J. D. Anderson, *Introduction to Flight*, Ch. 6 است.

۱. مسئله

یک مهندس آزمون پرواز فایلی CSV به شما می‌دهد: پنج دقیقه از پرواز آزمایشی یک پهپاد بال‌ثابت کوچک. شتاب‌سنج و ژیروسکوپ سه‌محوره با نرخ ۵۰ هرتز، به‌همراه ارتفاع بارومتری، سرعت هوایی پیتوت و دمای هوای بیرون ثبت شده‌اند. مهندس ارشد می‌پرسد:

1. **پرسش ۱.** نرخ نمونه‌برداری چقدر بود و آیا لاگر داده‌ای از دست داد؟
2. **پرسش ۲.** بیشینه‌ی ضریب بار چقدر بود؟ آیا از حد سازه‌ای ۲٫۰g عبور شد؟
3. **پرسش ۳.** نویز ژیروسکوپ‌ها هنگام توقف هواپیما چقدر است؟
4. **پرسش ۴.** بیشینه‌ی نرخ صعود چقدر بود؟

این‌ها هنوز پرسش‌های یادگیری ماشین نیستند، اما هر مدلی که بعدها در این درس می‌سازیم پاسخ آن‌ها را به ارث می‌برد. مدلی که روی داده‌ای با گام زمانی نامعلوم، شکاف‌های علامت‌نخورده یا خارهای پاک‌نشده آموزش ببیند، همان عیوب را یاد می‌گیرد. بنابراین هفته‌ی ۱ عادت‌ها و ابزارهای نگاه صادقانه به داده را می‌سازد: محیط تکرارپذیر، Pandas، NumPy و Matplotlib.

۲. محیط تکرارپذیر

نتیجه‌ای که فقط روی لپ‌تاپ نویسنده اجرا شود، نتیجه نیست. *محیط ایزوله* یک نصب خصوصی پایتون با نسخه‌های مستقل بسته‌هاست تا دو پروژه (یا دو دانشجو) هرگز مزاحم یکدیگر نشوند. دو ابزار برای ساخت آن داریم:

- **conda**: با دستورهای `conda create -n aero-ai python=3.11` و `conda activate aero-ai`؛ خود پایتون و کتابخانه‌های کامپایل‌شده را مدیریت می‌کند و از هفته‌ی ۹ برای PyTorch به آن نیاز داریم.
- **venv**: با `python -m venv .venv` و سپس `pip install -r requirements.txt`؛ درون خود پایتون است و سبک‌تر.

«دستور پخت» محیط یک فایل متنی (`environment.yml` یا `requirements.txt`) با نسخه‌های تثبیت‌شده است که کنار کد در گیت نگهداری می‌شود. نوت‌بوک‌های **Jupyter** کد، نتیجه و توضیح را کنار هم می‌گذارند و برای آزمایشگاه مناسب‌اند؛ هر چیزی که دوباره استفاده می‌شود در ماژول‌های `.py` قرار می‌گیرد. **Git** همه‌ی نسخه‌های کار را ثبت می‌کند:

`git switch -c` شاخه می‌سازد، `git add` تغییرات را آماده می‌کند، `git commit` آن‌ها را ثبت و `git push` منتشر می‌کند. اجرای `git status` پیش از هر commit ارزان‌ترین عادت پیشگیری از خطا در این درس است.

۳. NumPy: آرایه، محور و برداری‌سازی

لیست پایتون ارجاع‌هایی به اشیای جداگانه نگه می‌دارد؛ اما `ndarray` در NumPy یک بلوک پیوسته از اعداد هم‌نوع است، پس عملیات روی کل آرایه در کد کامپایل‌شده اجرا می‌شود. در آزمایشگاه، محاسبه‌ی اندازه‌ی ۲۰۰٬۰۰۰ بردار شتاب با `np.sqrt(np.sum(a**2, axis=1))` حدود ده برابر سریع‌تر از حلقه است.

برای لاگی با شکل $(N, 3)$، محور ۰ روی نمونه‌های زمانی و محور ۱ روی مؤلفه‌های x، y، z است. کاهش روی `axis=0` برای هر مؤلفه یک مقدار می‌دهد (مثلاً بایاس هر ژیرو) و کاهش روی `axis=1` برای هر نمونه یک مقدار (مثلاً اندازه‌ی بردار). **انتشار (broadcasting)** اجازه می‌دهد شکل‌های $(N, 3)$ و $(3,)$ با هم ترکیب شوند: بردار روی همه‌ی سطرها اعمال می‌شود، پس `a - a.mean(axis=0)` بدون حلقه بایاس هر محور را حذف می‌کند.

مثال حل‌شده‌ی ۱: کدام انحراف معیار؟

پنج نمونه‌ی ژیرو در حالت سکون: $x = [0.2,\ -0.4, 0.1, 0.5,\ -0.4]$ درجه بر ثانیه. میانگین $\bar{x} = 0$ است و مجموع مربعات انحراف برابر $0.04 + 0.16 + 0.01 + 0.25 + 0.16 = 0.62$.

$$\sigma = \sqrt{0.62/5} = 0.352, \qquad s = \sqrt{0.62/4} = 0.394$$

تابع `np.std(x)` مقدار σ (با `ddof=0`) و `pd.Series(x).std()` مقدار s (با `ddof=1`) را برمی‌گرداند: یک داده، دو نتیجه با ۱۲ درصد اختلاف. برای مشخصه‌ی نویز، s برآورد درست است (واریانس نااریب از یک نمونه)؛ در هر دو کتابخانه `ddof=1` را صریح بنویسید.

۴. ضریب بار از روی شتاب‌سنج

شتاب‌سنج **نیروی ویژه** $\mathbf{f}$ را می‌سنجد: نیروی غیرگرانشی بر واحد جرم. روی زمین و در حالت توقف، $+g$ رو به بالا نشان می‌دهد. ضریب بار برابر است با

$$n = \frac{\|\,\mathbf{f}\,\|}{g_0}, \qquad g_0 = 9.80665\ \mathrm{m/s^2}.$$

در دور هماهنگ تراز با زاویه‌ی بانک φ، مؤلفه‌ی قائم برآ وزن را خنثی می‌کند ($L\cos\varphi = W$)، پس

$$n = \frac{L}{W} = \frac{1}{\cos\varphi}.$$

بانک ۳۰ درجه ضریب بار ۱٫۱۵۵ می‌دهد و حد ۲٫۰g معادل بانک ۶۰ درجه است.

مثال حل‌شده‌ی ۲: یک نمونه با دست

خوانش $\mathbf{f} = (1.2,\ -0.4,\ 19.6)$ متر بر مجذور ثانیه است. پس $\|\,\mathbf{f}\,\| = 19.641$، $\|\,\mathbf{f}\,\|^2 = 1.44 + 0.16 + 384.16 = 385.76$ و $n = 19.641/9.80665 = 2.003$. یک نمونه بالای حد است؛ اینکه *بدنه* واقعاً از حد عبور کرده یا نه، پرسش جداگانه‌ای است (بخش ۶).

۵. Pandas: خواندن داده مثل یک مهندس

`pd.read_csv` یک `DataFrame` برمی‌گرداند: جدولی از ستون‌های نام‌دار NumPy. سه فراخوانی تصویر اولیه را می‌دهند: `head()` (آیا فایل درست به نظر می‌رسد؟)، `info()` (نوع داده‌ها و تعداد مقادیر غیرتهی) و `describe()` (تعداد، میانگین، انحراف معیار، کمینه، چارک‌ها و بیشینه‌ی هر ستون). در لاگ ما، `describe()` همین حالا دو مشکل را نشان می‌دهد: ستون‌های `baro_alt_m` و `airspeed_mps` ۷۵ مقدار کمتر از بقیه دارند (داده‌ی گمشده)، و بیشینه‌ی `accel_z_mps2` برابر ۵۰٫۳ است، یعنی حدود ۵g.

۶. پاسخ به چهار پرسش

پرسش ۱، نمونه‌برداری و شکاف‌ها. نرخ نمونه‌برداری $f_s = 1/\operatorname{median}(\Delta t)$ است. میانه مهم است: در بُرش $t = [99.94, 99.96, 99.98, 101.20, 101.22]$ اختلاف‌ها $0.02, 0.02, 1.22, 0.02$ ثانیه‌اند. میانگین آن‌ها (۰٫۳۲ ثانیه) با یک شکاف خراب می‌شود، اما میانه (۰٫۰۲ ثانیه) همان ۵۰ هرتز درست را می‌دهد. فرکانس نایکوئیست ۲۵ هرتز است. علامت‌گذاری هر گام بزرگ‌تر از ۱٫۵ برابر گام اسمی، یک شکاف با $1.22/0.02 - 1 = 60$ نمونه‌ی گمشده را پیدا می‌کند، و `df.isna().mean()` نشان می‌دهد ۰٫۵ درصد مقادیر بارومتر و سرعت هوایی NaN است.

پرسش ۲، خار یا مانور. بیشینه‌ی خام ۵٫۱۳g در لحظه‌ی ۶۶٫۶۸ ثانیه است، اما همه‌ی همسایه‌هایش ۱٫۰۶g هستند: رخدادی تک‌نمونه‌ای (۲۰ میلی‌ثانیه) یک خطای حسگر است، چون مانور واقعی در چند ثانیه شکل می‌گیرد. **میانه‌ی متحرک** مرکزی چنین خارهایی را حذف و مانورها را حفظ می‌کند.

مثال حل‌شده‌ی ۳. برای پنجره‌ی $[1.00, 1.01, 5.10, 0.99, 1.00]$ میانگین $9.10/5 = 1.82$ است (خار نفوذ می‌کند)، اما میانه‌ی پنجره‌ی مرتب‌شده‌ی $[0.99, 1.00, 1.00, 1.01, 5.10]$ برابر ۱٫۰۰ است.

پس از میانه‌ی متحرک ۵ نمونه‌ای، بیشینه‌ی واقعی **۱٫۸۰g** در مانور بالاکشیدن حدود ثانیه‌ی ۱۹۲ است: از حد ۲٫۰g عبور نشده است.

پرسش ۳، نویز در حالت سکون. انتخاب $t < 25$ با ماسک بولی نشان می‌دهد هر سه ژیرو بایاسی کمتر از ۰٫۰۲ و انحراف معیار نمونه‌ای حدود **۰٫۳۰ درجه بر ثانیه** دارند و هیستوگرام‌ها تقریباً گاوسی‌اند (نویز سفید). این عدد ورودی فیلتر کالمن تیم ناوبری است.

پرسش ۴، نرخ صعود. مشتق‌گیری نویز را تشدید می‌کند.

مثال حل‌شده‌ی ۴. با $h = [250.0, 250.3, 250.5]$ متر در $t = [80.00, 80.02, 80.04]$ ثانیه، تفاضل مرکزی برابر است با

$$\dot{h}(80.02) \approx \frac{h(80.04) - h(80.00)}{0.04} = 12.5 \text{ m/s}.$$

اگر هر نمونه نویز بارومتری $\sigma_h = 0.8$ متر داشته باشد، صورت کسر انحراف معیار $\sqrt{2}\,\sigma_h = 1.13$ متر دارد و برآورد حدود ±۲۸ متر بر ثانیه نویز خواهد داشت. پس ابتدا ارتفاع را روی شبکه‌ی زمانی یکنواخت ۵۰ هرتز می‌بریم (NaNها و شکاف با درون‌یابی خطی پر می‌شوند)، با پنجره‌ی مرکزی ۲ ثانیه‌ای هموار می‌کنیم و فقط پس از آن مشتق می‌گیریم. بیشینه‌ی نرخ صعود حدود **۱۶ متر بر ثانیه** در حدود ثانیه‌ی ۸۴ است که با مقدار طراحی ۱۵ متر بر ثانیه سازگار است.

۷. Matplotlib و گام به سوی کلاس‌ها

`fig, axes = plt.subplots(4, 1, sharex=True)` پنل‌هایی هم‌تراز با محور زمانی مشترک می‌سازد: ارتفاع، سرعت هوایی، ضریب بار با خط‌چین ۲٫۰g، و نرخ صعود؛ شکاف لاگر با `axvspan` سایه زده می‌شود. هر محور یکا دارد و هر پنل به یک پرسش پاسخ می‌دهد.

آزمایشگاه با تکرار هر چهار پاسخ به کمک کلاس `FlightLog` در `src/aerospace_ai/utils.py` پایان می‌یابد. این کلاس همان منطق را با سازنده‌ی `@classmethod` (`from_csv`)، ویژگی‌های محاسبه‌ای `@property` (`sample_rate_hz`)، `__slots__`، `__repr__`، نوع‌دهی و docstring بسته‌بندی می‌کند. هفته‌ی ۲ با سلسله‌مراتب کلاس `Sensor` بر همین پایه بنا می‌شود.

آنچه اکنون بلدیم انجام دهیم

- ساخت، فعال‌سازی و تثبیت نسخه‌های محیط؛ شاخه‌سازی، commit و push با گیت.
- جایگزینی حلقه با NumPy برداری، انتخاب `axis` درست و استفاده از broadcasting.
- بارگذاری CSV حسگر و خواندن نقادانه‌ی `head`، `info` و `describe`.
- یافتن مقاوم نرخ نمونه‌برداری، شکاف‌ها و مقادیر گمشده.
- جدا کردن خار حسگر از مانور واقعی با میانه‌ی متحرک.
- تخمین نویز (با ذکر `ddof`) و مشتق (اول هموارسازی) از اندازه‌گیری‌های نویزی.

تمرین: مسائل کلاسی `in_class_problems/week01_problems.ipynb`، آزمایشگاه `notebooks/week01/week01_python_env.ipynb` و تمرین ۰ (`assignments/assignment00/`).